WESTERN KENTUCKY UNIVERSITY
Department of Computer Science
CS 425 — Operating Systems I

Shared-Memory Multiprocessing for Producer–Consumer Coordination

PROJECT 1

Prepared by

Yahir Perez
Clement Anzuruni

November 4, 2026

Abstract

We implemented a bounded producer–consumer buffer using separate C processes and POSIX shared memory. Producers place integers into a shared circular buffer, and consumers remove and print them. The code deliberately omits synchronization mechanisms, as required by the assignment. The experiment examines both execution performance and the errors that can arise when processes modify the same variables without coordination.

We ran 27 trials on a MacBook Pro with an Apple M4 Pro processor. The nine configurations varied buffer capacity or the number of producer or consumer processes. Mean elapsed times ranged from 5.695 to 8.013 ms, and core-normalized CPU utilization ranged from 16.02% to 30.70%. Every trial contained both duplicated and missing output values. Higher CPU use did not consistently correspond to faster completion, and the varying output shows why timing must be considered together with correctness.

1. Objectives and scope

The objectives are to (1) place only necessary cross-process state in shared memory, (2) launch and reap producer and consumer processes with fork() and execlp(), (3) measure elapsed time and aggregate CPU usage, and (4) study how configuration changes and race conditions affect observed behavior. This report describes the C implementation and measurements collected on the MacBook Pro.

2. Implementation and shared-memory design

The main.c program validates four command-line arguments, creates and sizes a POSIX shared-memory object with shm_open() and ftruncate(), maps it with mmap(MAP_SHARED), initializes the state, then starts producer and consumer executables using fork() and execlp(). Each child independently opens and maps the same named object. The parent waits for all producers, marks production_done, waits for consumers, captures resource usage with wait4(), and releases the mapping with munmap() and shm_unlink().

Variable(s)	Placement	Reason
buffer[]	Shared	Producers write and consumers read the same bounded storage.
count	Shared	Both sides inspect/update the believed number of buffered items.
head and tail	Shared	Consumers use head; producers use tail to choose circular-buffer slots.
next_value	Shared	Every producer reads/updates one common integer source.
production_done	Shared	The parent signals when all producers have exited; consumers inspect it.
capacity, upper_limit	Process-local	Configuration is passed to children through execlp() arguments.
value, slot, observed_count	Process-local	Temporary calculations belong only to the current child.
PIDs, file descriptors, timestamps, rusage	Process-local	Process creation, resource handling, and measurement are managed locally.

2.1 Circular-buffer operations

A producer reads next_value and checks that it is not above upper_limit. If count equals capacity, it yields and retries. Otherwise it writes the value at tail, advances tail modulo capacity, updates count, and sets next_value to

value + 1. A consumer checks count, reads the slot at head when it believes the buffer is nonempty, advances head modulo capacity, decrements count, and prints the value. The check/read/write/update sequence is deliberately not atomic.

2.2 Termination and cleanup

Each producer exits after observing next_value > upper_limit. The parent sets production_done = 1 after waiting for all producers. Consumers exit only when they observe both count == 0 and production_done. The parent reaps every created child using wait4(), then unmaps and unlinks the shared-memory object. Since the unsynchronized occupancy count can be stale or corrupted, these stopping conditions do not guarantee exactly-once processing or termination in every possible interleaving. A test-script timeout provides an external safeguard during experiments.

The shared integer fields are declared volatile and retries call sched_yield(); neither provides mutual exclusion nor makes compound updates atomic.

3. Experimental setup and measurement

All 27 experimental trials were run on one Apple MacBook Pro using an Apple M4 Pro processor, 24 GB unified memory, macOS Tahoe 26.6.2, and 14 logical CPU cores as reported by the program. Compilation used Apple Clang 21.0.0 with the Makefile flags -std=c11 -O0 -Wall -Wextra -Wpedantic. For every trial, consumer output was captured by the same Python script rather than displayed live, keeping the output policy consistent.

The baseline was buffer size B = 8, producers P = 2, consumers C = 2, and upper limit = 5,000 (intended integers 0 through 5,000, inclusive). Nine configurations were evaluated, with three repetitions of each. The baseline was rerun in each parameter sweep rather than reused from a different sweep.

Sweep	Parameter values	Other settings held constant
Buffer size	B = 2, 8, 32	P = 2, C = 2, upper limit = 5000
Producer count	P = 1, 2, 4	B = 8, C = 2, upper limit = 5000
Consumer count	C = 1, 2, 4	B = 8, P = 2, upper limit = 5000

3.1 Performance measures

Elapsed wall-clock time was measured using CLOCK_MONOTONIC from immediately before process creation until all producer and consumer processes were reaped. Parent CPU time was determined by the difference in getrusage(RUSAGE_SELF) readings, and each child's user + system time was added exactly once from wait4().

Core-normalized CPU utilization: $U_{CPU} = 100 \times T_{CPU} / (C \times T_{elapsed})$, where C = 14 logical CPU cores, T_{CPU} includes the measured parent and all children, and $T_{elapsed}$ is wall-clock time. The results use means and sample standard deviations (SD, n = 3).

4. Summary of measured results

Table 1. Summary of nine tested configurations. Duplicate and missing counts describe output behavior, not a controlled amount of correct work.

Variable setting	Time, ms (mean +/- SD)	CPU % (mean +/- SD)	Mean prints	Mean dup.	Mean missing
Buffer = 2	7.151 +/- 0.601	21.80 +/- 0.46	5483.3	842.3	360.0
Buffer = 8	6.354 +/- 0.732	21.01 +/- 1.02	5233.3	679.0	446.7
Buffer = 32	5.695 +/- 0.117	21.36 +/- 0.10	5215.7	385.0	170.3

Variable setting	Time, ms (mean +/- SD)	CPU % (mean +/- SD)	Mean prints	Mean dup.	Mean missing
Producers = 1	6.065 +/- 0.612	16.02 +/- 0.30	4968.7	456.3	488.7
Producers = 2	5.996 +/- 0.336	21.45 +/- 0.23	5194.7	647.7	454.0
Producers = 4	6.642 +/- 0.443	30.70 +/- 0.84	5079.3	613.0	534.7
Consumers = 1	6.341 +/- 0.986	16.54 +/- 0.65	4928.7	221.7	294.0
Consumers = 2	5.836 +/- 0.336	21.23 +/- 0.33	5289.3	694.0	405.7
Consumers = 4	8.013 +/- 0.368	28.98 +/- 0.27	5121.3	712.3	592.0

5. Buffer-size figures

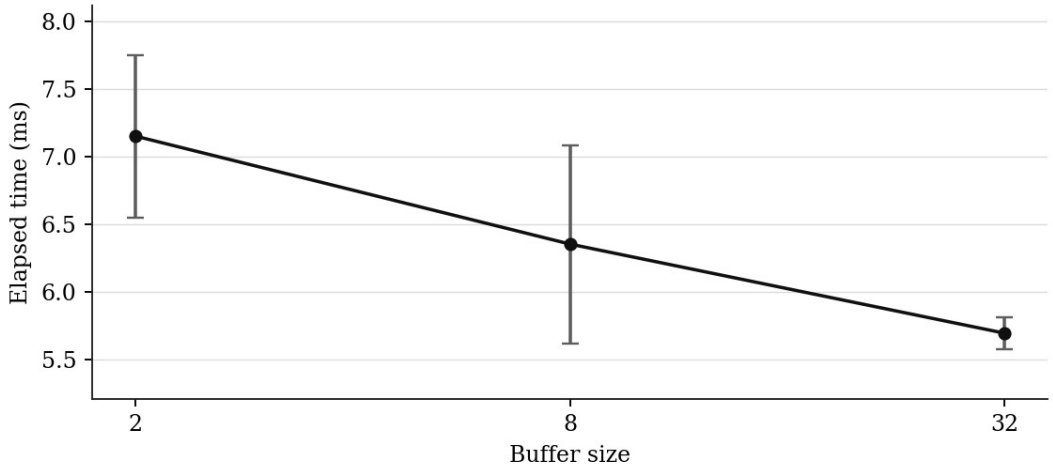


Figure 1. Elapsed time vs. buffer size (mean +/- 1 sample SD; n = 3).

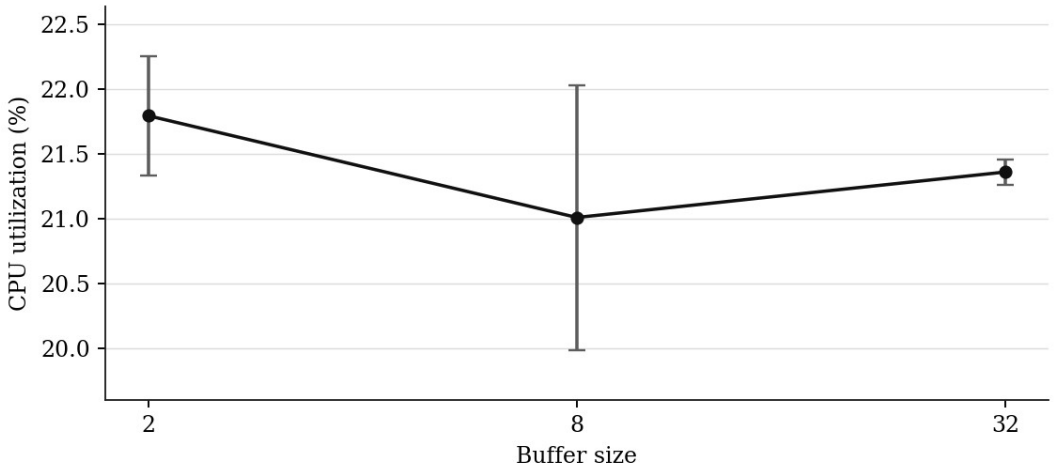


Figure 2. CPU utilization vs. buffer size (mean +/- 1 sample SD; n = 3).

6. Producer-count figures

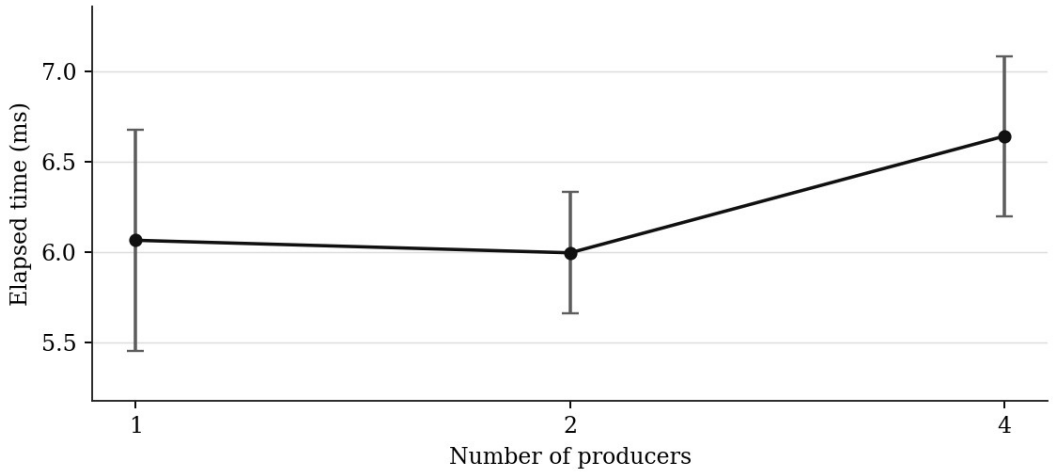


Figure 3. Elapsed time vs. number of producers (mean +/- 1 sample SD; n = 3).

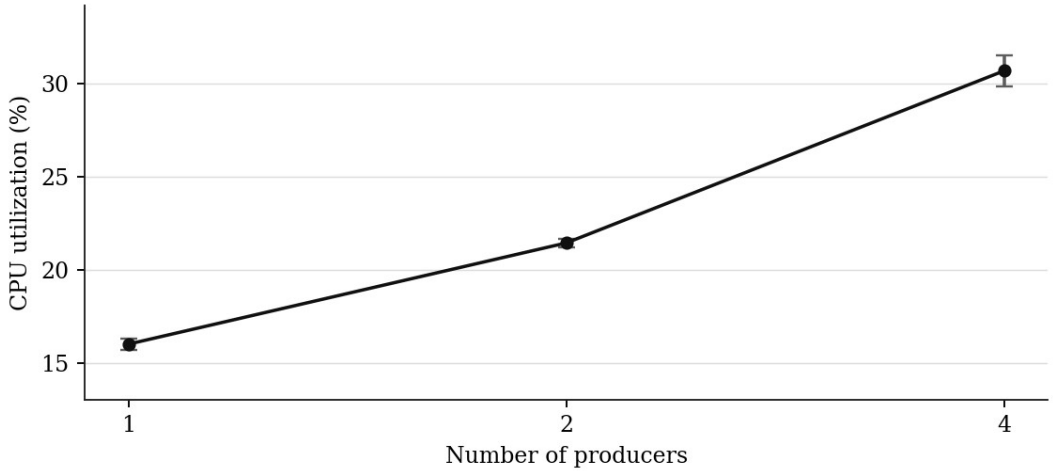


Figure 4. CPU utilization vs. number of producers (mean +/- 1 sample SD; n = 3).

7. Consumer-count figures

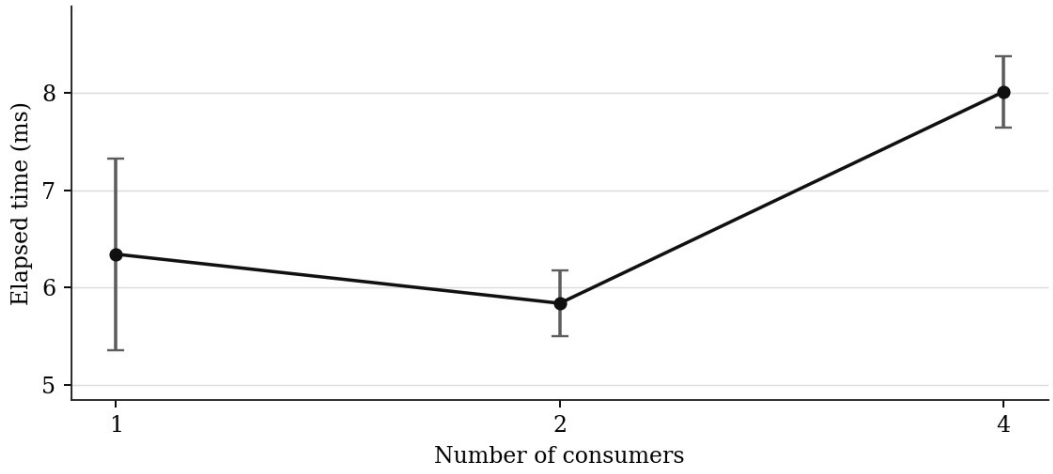


Figure 5. Elapsed time vs. number of consumers (mean +/- 1 sample SD; n = 3).

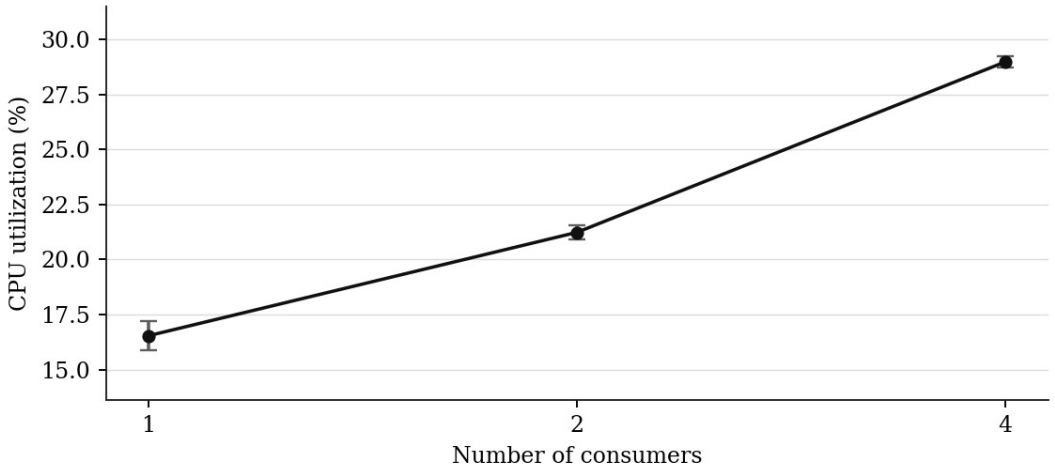


Figure 6. CPU utilization vs. number of consumers (mean +/- 1 sample SD; n = 3).

8. Performance analysis

8.1 Buffer capacity (B = 2, 8, 32)

With two producers and two consumers, mean elapsed time decreased from 7.151 ms at B = 2 to 6.354 ms at B = 8 and 5.695 ms at B = 32. The decrease from B = 2 to B = 32 was about 20.4%. CPU utilization was broadly similar (21.80%, 21.01%, and 21.36%). Larger buffer space may reduce time spent retrying after a full-buffer observation. However, printed counts and correctness varied across trials; the timing difference alone does not establish improved useful throughput.

8.2 Producer count (P = 1, 2, 4)

At B = 8 and C = 2, mean elapsed times were 6.065, 5.996, and 6.642 ms for one, two, and four producers. The slight difference between one and two producers is small compared with observed run-to-run variation, and four producers were slower on average. CPU utilization rose from 16.02% to 21.45% to 30.70%, consistent with more runnable processes consuming CPU time without guaranteed speedup. Multiple producers also compete to update next_value, count, and tail.

8.3 Consumer count (C = 1, 2, 4)

At B = 8 and P = 2, mean elapsed times were 6.341, 5.836, and 8.013 ms. Two consumers had the shortest mean, whereas four had the longest; four consumers took about 26.4% longer than one on average. CPU utilization rose from 16.54% to 21.23% to 28.98%. Extra consumer activity may add scheduling and shared-state contention, and extra printing activity does not necessarily represent useful work.

8.4 Interpretation

Figure error bars represent one sample SD, not confidence intervals. With only three trials and runtimes around 5–8 ms, start-up costs, process scheduling, machine activity, and changes in the number of output values can materially influence differences. The study therefore characterizes this implementation under these settings; it does not support a general claim that any hardware configuration or process count is universally optimal.

9. Race-condition detection and analysis

The script checked each run's printed values against the intended set {0, 1, ..., 5000}; it counted all printed lines, occurrences beyond the first (duplicates), and expected values not present (missing). The same configurations were

repeated three times to identify variation across interleavings. Every one of the 27 runs showed nonzero duplicate and missing counts.

Table 2. Output anomalies were present in every three-trial group. Each row summarizes a separate sweep configuration.

Parameter setting	Mean duplicate values	Mean missing values	Mean printed values
B = 2	842.3	360.0	5483.3
B = 8	679.0	446.7	5233.3
B = 32	385.0	170.3	5215.7
P = 1	456.3	488.7	4968.7
P = 2	647.7	454.0	5194.7
P = 4	613.0	534.7	5079.3
C = 1	221.7	294.0	4928.7
C = 2	694.0	405.7	5289.3
C = 4	712.3	592.0	5121.3

9.1 Plausible producer interleaving

Two producers can both observe count = 0, tail = 0, and next_value = 12 before either writes. They may both write buffer[0] = 12, advance the same tail position, and store count = 1. Two logical insert attempts thus collapse into one observed slot or count update. A similar lost update can occur when each producer writes next_value = 13. A value can be produced twice while another later value is skipped.

9.2 Plausible consumer interleaving

Two consumers can both observe a nonzero count and the same head index before either advances it. Both read and print the same slot; their overlapping count/head writes may lose a decrement or skip subsequent data. A producer/consumer overlap can also overwrite a slot before its previous value is consumed. These sequences are plausible mechanisms consistent with duplicates and missing values; the logs do not identify an exact instruction interleaving for each anomaly.

10. Limitations and conclusion

10.1 Limits of the experiment

- Unsynchronized shared state: read-check-update sequences are not atomic; count may not reflect the actual contents of the buffer, and a zero final count alone does not prove correct delivery.
- Termination: production_done is set after producer completion, but it is also read without an explicit synchronization guarantee; incorrect occupancy can lead to early exit, missing work, or a possible hang under other schedules.
- Very short runs: tests lasting roughly 5-8 ms are sensitive to process startup, scheduler timing, and uncontrolled background work. Only three samples were taken per configuration.
- Different effective workloads: the number of values printed differed across configurations and runs, so lower elapsed time is not directly comparable to higher correct throughput.
- Single platform and compiler setting: results describe this Apple Silicon Mac and this build/output policy, and should not be generalized to other machines without additional controlled experiments.

10.2 Conclusion

The project successfully demonstrates a multi-process bounded buffer using POSIX shared memory and records elapsed time and core-normalized CPU utilization without using synchronization primitives. Increasing buffer capacity was associated with lower mean elapsed time in this sample, while increasing the number of producers or consumers substantially increased CPU usage without consistent speedup. Repeated duplicate and missing values reveal the correctness cost of the deliberately unsynchronized design. These observations support the central lesson: process parallelism and high CPU utilization do not guarantee correct or faster useful work.

References

[1] CS 425, “Project 1: Shared-Memory Multiprocessing for Producers–Consumers Coordination,” assignment handout, Fall 2026.

Appendix A. Individual Mac experiment trials

Elapsed time is shown in milliseconds (ms). CPU utilization is normalized by the 14 logical cores. These 27 measurements were transcribed from the recorded macOS experiments.py console output; all values are from the same Mac test session.

Table A1. Per-trial elapsed time, utilization, and printed-value checks on macOS Tahoe 26.6.2.

Sweep	B/P/C	Trial	Time (ms)	CPU (%)	Printed	Dup.	Missing
Buffer	2/2/2	1	7.717	21.267	5532	858	327
Buffer	2/2/2	2	7.216	22.001	5615	955	341
Buffer	2/2/2	3	6.521	22.120	5303	714	412
Buffer	8/2/2	1	5.844	21.305	5063	525	463
Buffer	8/2/2	2	7.193	19.873	5058	629	572
Buffer	8/2/2	3	6.024	21.854	5579	883	305
Buffer	32/2/2	1	5.805	21.349	5462	559	98
Buffer	32/2/2	2	5.573	21.272	5059	303	245
Buffer	32/2/2	3	5.708	21.464	5126	293	168
Producers	8/1/2	1	5.831	15.932	4923	446	524
Producers	8/1/2	2	6.759	16.348	4987	483	497
Producers	8/1/2	3	5.605	15.765	4996	440	445
Producers	8/2/2	1	5.721	21.682	4929	515	587
Producers	8/2/2	2	5.895	21.469	5184	690	507
Producers	8/2/2	3	6.371	21.213	5471	738	268
Producers	8/4/2	1	6.381	31.276	5112	638	527
Producers	8/4/2	2	6.392	31.100	5108	631	524
Producers	8/4/2	3	7.154	29.737	5018	570	553
Consumers	8/2/1	1	7.472	15.793	4886	326	441
Consumers	8/2/1	2	5.662	16.863	4937	111	175
Consumers	8/2/1	3	5.890	16.961	4963	228	266
Consumers	8/2/2	1	5.549	20.990	4976	500	525
Consumers	8/2/2	2	5.754	21.106	5254	646	393

Sweep	B/P/C	Trial	Time (ms)	CPU (%)	Printed	Dup.	Missing
Consumers	8/2/2	3	6.205	21.602	5638	936	299
Consumers	8/2/4	1	7.643	29.216	5233	665	433
Consumers	8/2/4	2	8.379	28.684	5140	801	662
Consumers	8/2/4	3	8.016	29.025	4991	671	681

Appendix B. Source-code organization

main.c: command-line validation, shared-memory creation, process management, timing, CPU accounting, cleanup. producer.c: insertion and shared next_value update. consumer.c: removal and printing. shared.h: shared memory layout and size calculation. experiments.py: repeatable trial execution, captured output analysis, summary statistics, and figure generation.